HERALD COLLEGE KATHMANDU

Week 3

Bash Scripting & Automation

Automating the build, push, and deploy of your MERN app · Tutorial 2 hrs · Workshop 2.5 hrs · Phase 2

You have built images and run containers by hand. This week you stop typing the same commands over and over and let a script do it. You will learn just enough bash to write two real automation scripts: one that builds your MERN image and pushes it to Docker Hub, and one that pulls that image onto an AWS EC2 server and runs it.

> **READ FIRST — WINDOWS USERS**
>
> **Bash does not run natively on Windows, but you already have it.** When you installed Docker Desktop in Week 1 you enabled **WSL2** (Windows Subsystem for Linux) — a real Linux environment inside Windows. Open the `Ubuntu` app from your Start menu, and you have a genuine bash shell where every command and script in this document runs identically to macOS and Linux. Do all of this week's work inside WSL2, not in PowerShell or Command Prompt.
>
> If WSL2 is somehow missing, open PowerShell *as Administrator* and run `wsl --install`, then restart. Everything else follows.

By the end of this week you can

— write and run a bash script, using variables, and make it fail safely;
— explain why automation beats typing commands by hand;
— write a script that builds a Docker image and pushes it to Docker Hub;
— launch an EC2 instance, install Docker on it, and connect over SSH;
— write a script that deploys your image from Docker Hub onto that EC2 server;
— diagnose the common scripting, authentication, and deployment failures.

1 Why Automate

Publishing a new version of your app means running the same four commands, in the same order, every time: build the image, tag it, log in, push it. Do it by hand and eventually you will mistype one, skip one, or run them out of order — and the release breaks in a way that is annoying to debug.

A script solves this permanently. It is nothing more than those commands saved in a file. You run the file once; it runs the commands for you, identically, every time.

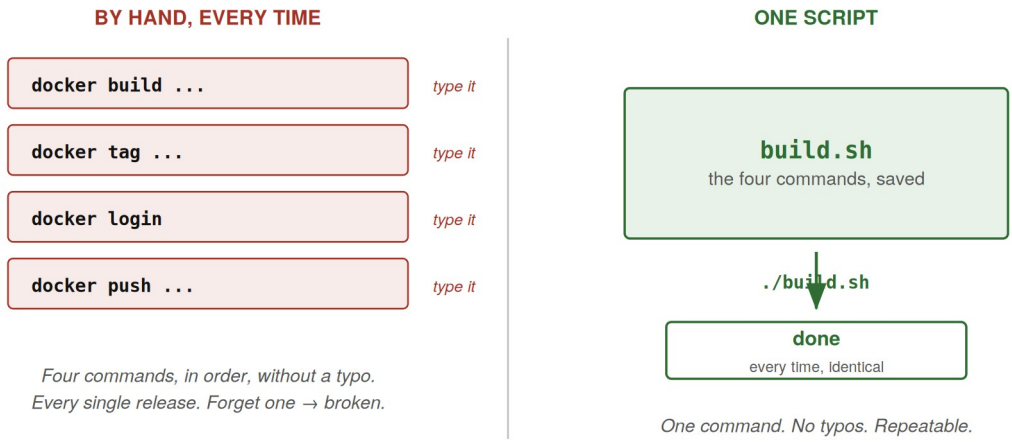

Figure 1. A script is the sequence of commands you already type — saved once, run reliably ever after.

This is the heart of DevOps: any task you do more than a couple of times should be automated, so it is fast, repeatable, and impossible to get subtly wrong.

2 Bash Essentials

Bash is the language of the Linux command line. A bash **script** is a text file of commands that bash runs top to bottom. You already know many of the commands — this section adds just the structure needed to put them in a script.

2.1 Your first script

Create a file called `hello.sh`:

```
#!/bin/bash
echo "Hello from a script"
```

The first line, `#!/bin/bash`, is called the **shebang**. It tells the system to run this file with bash. Make the file executable and run it — the same `chmod` you met in Week 1:

```
chmod +x hello.sh
./hello.sh
```

2.2 Variables

A variable stores a value so you can define it once and reuse it. Assign with `name=value` (no spaces around the =), and read it back with a `$` in front:

```
#!/bin/bash
IMAGE="mern-app"
echo "Building $IMAGE"          # prints: Building mern-app
```

Always wrap variables in double quotes — `"$IMAGE"` — when you use them. It prevents a whole class of bugs when a value is empty or contains spaces.

2.3 Making a script fail safely

This one line prevents most scripting disasters. By default, if one command in a script fails, bash carries on to the next anyway — so a failed build is followed by a push of a stale image, and you never notice. Put this near the top of every script:

```
set -euo pipefail
```

Part	What it does
`-e`	Exit immediately if any command fails
`-u`	Treat use of an undefined variable as an error
`-o pipefail`	Make a pipeline fail if any command in it fails

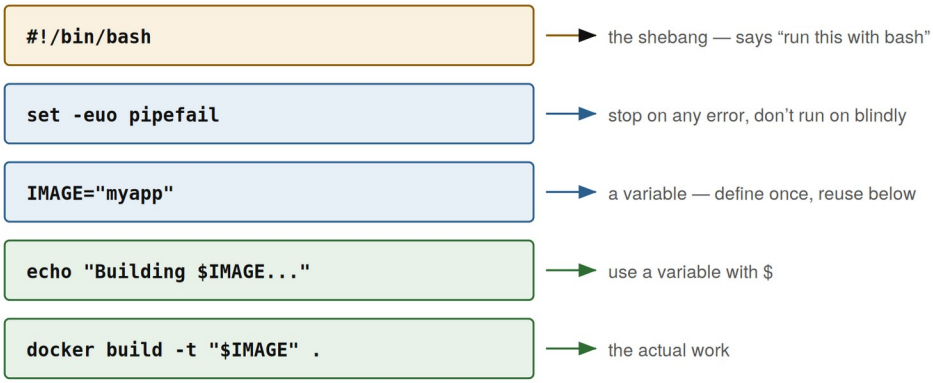

Figure 2. The anatomy of a safe script: shebang, safety line, variables, then the work.

> **A NOTE ON SECRETS**
>
> Never write passwords or access keys directly in a script — anyone who reads the file gets them, and they end up in Git history. This week we log in to Docker Hub interactively, and later use SSH keys for EC2. Keeping credentials out of scripts is a habit worth forming now.

3 Script 1: Build and Push to Docker Hub

Now the first real script. It builds your MERN image and publishes it to **Docker Hub**, the image registry from Week 2. From there, any machine — including your EC2 server — can pull it.

3.1 How pushing to Docker Hub works

Docker Hub organises images by username. To push, an image must be named `YOUR_USERNAME/repository:tag`. The workflow is always the same four steps:

Step	Command
`Build`	docker build -t NAME .
`Tag`	docker tag NAME USER/NAME:TAG
`Log in`	docker login
`Push`	docker push USER/NAME:TAG

Create a free account at `hub.docker.com` if you have not already, then log in once from your terminal so the script does not have to handle your password:

```
docker login
# enter your Docker Hub username and password once;
# Docker remembers it for future pushes
```

3.2 The script

In your project's `server` folder (the backend from Week 2), create `build-and-push.sh`. Read the comments — each line maps to a step you already know.

build-and-push.sh

```
#!/bin/bash
set -euo pipefail

# --- settings (change USERNAME to your Docker Hub username) ---
USERNAME="your-dockerhub-username"
IMAGE="mern-server"
TAG="v1"

FULL_NAME="$USERNAME/$IMAGE:$TAG"

# --- 1. build the image from the Dockerfile here ---
echo "Building $FULL_NAME ..."
docker build -t "$FULL_NAME" .

# --- 2. push it to Docker Hub ---
echo "Pushing $FULL_NAME ..."
docker push "$FULL_NAME"

echo "Done. Image is live on Docker Hub."
```

Notice we build straight to the full `USER/IMAGE:TAG` name with `-t`, which saves a separate `docker tag` step. Run it:

```
chmod +x build-and-push.sh
./build-and-push.sh
```

When it finishes, open `hub.docker.com` in a browser and you will see your image in your repositories. It is now downloadable from anywhere.

4 Preparing an EC2 Server

An EC2 instance is a virtual Linux server rented from AWS. This is where your image will run so that it is reachable on the internet, not just on your laptop. Preparing it is a one-time setup, done by hand in the AWS console and over SSH. It needs just one thing installed: Docker. After that, the deploy script in Section 5 drives it remotely.

4.1 Launch the instance

In the AWS console, launch an EC2 instance with these choices:

- **AMI:** Amazon Linux 2023 (the current default).
- **Type:** t2.micro or t3.micro — both free-tier eligible.
- **Key pair:** create one and download the `.pem` file. This is your SSH key — keep it safe; you cannot download it again.
- **Security group:** allow SSH (port 22) so you can connect, and add a rule for the port your app uses (for example 4200) so the app is reachable.

4.2 Connect over SSH

From your terminal (WSL2 on Windows), connect using the key and the instance's public IP, shown in the console. The default username on Amazon Linux is `ec2-user`:

```
# restrict the key's permissions first, or SSH refuses it (Week 1)
chmod 400 my-key.pem

ssh -i my-key.pem ec2-user@<EC2_PUBLIC_IP>
```

4.3 Install Docker on the server

A fresh EC2 instance has no Docker. Once connected over SSH, install it. These are the current commands for Amazon Linux 2023:

```
sudo dnf update -y
sudo dnf install -y docker
sudo systemctl enable --now docker

# let ec2-user run docker without sudo
sudo usermod -aG docker ec2-user

# log out and back in so the group change takes effect
exit
```

Reconnect with the same `ssh` command, then confirm Docker works: `docker run hello-world`.

5 Script 2: Deploy to EC2 from Your Machine

Deploying is just pulling your image from Docker Hub and running it — the Week 1 skills, now on a remote server. The clean way to do this is **without logging in to the server by hand**. This script runs on **your own machine** and uses SSH to trigger the pull and run *on* EC2.

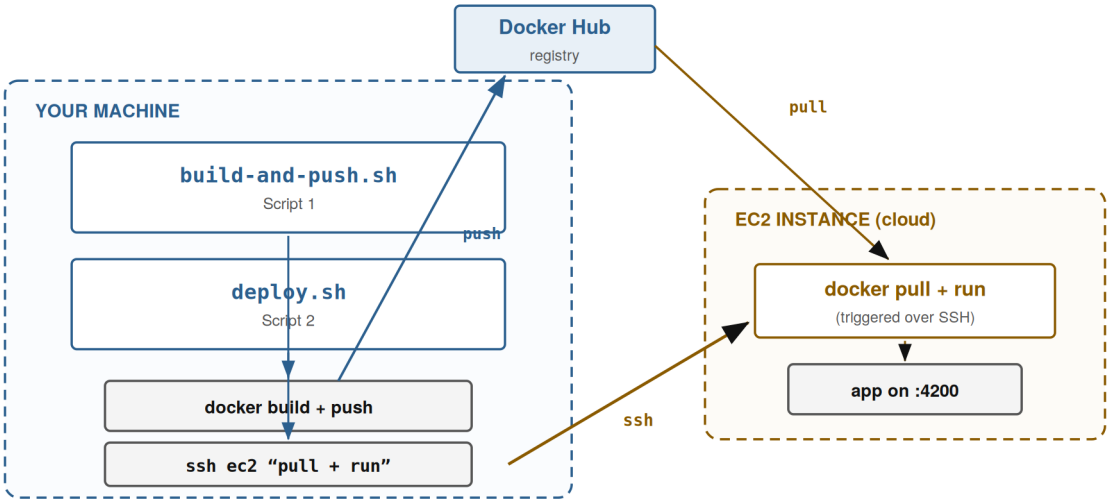

Figure 3. Both scripts run from your machine. Script 2 uses SSH to trigger the deploy commands on EC2.

5.1 Running a command on a remote server with SSH

The trick is that `ssh` can take a command as an argument. Instead of opening a session you sit inside, it runs the command **on the remote machine** and returns:

```
ssh -i my-key.pem ec2-user@<EC2_PUBLIC_IP> "docker ps"
# runs 'docker ps' on the EC2 server, prints the result, exits
```

Put several commands inside the quotes and they all run on the server, in order. That is the whole basis of the deploy script.

5.2 The script

On **your own machine**, in the `server` folder, create `deploy.sh`:

deploy.sh (runs on your machine)

```
#!/bin/bash
set -euo pipefail

# --- settings (same image you pushed in Script 1) ---
USERNAME="your-dockerhub-username"
IMAGE="mern-server"
TAG="v1"
FULL_NAME="$USERNAME/$IMAGE:$TAG"
CONTAINER="mern-server"

# --- connection details ---
KEY="my-key.pem"
EC2_HOST="ec2-user@<EC2_PUBLIC_IP>"

echo "Deploying $FULL_NAME to $EC2_HOST ..."
```

```
# run all the deploy commands ON the server, over SSH
ssh -o StrictHostKeyChecking=accept-new -i "$KEY" "$EC2_HOST" "
  docker pull $FULL_NAME
  docker stop $CONTAINER 2>/dev/null || true
  docker rm $CONTAINER 2>/dev/null || true
  docker run -d --name $CONTAINER \
    --restart always -p 4200:4200 $FULL_NAME
"

echo "Deployed. App is live on port 4200."
```

Everything inside the quotes runs on EC2; everything outside runs on your laptop. Three details are worth understanding:

— `|| true` — on the first deploy there is no old container to stop, so those commands fail. Because of `set -e` that would kill the script, so `|| true` says "if this fails, carry on."

— `--restart always` — tells Docker to restart the container automatically if it crashes or the server reboots. This is what keeps a deployment up.

— `-o StrictHostKeyChecking=accept-new` — the first time you SSH to a new server it normally stops to ask "are you sure?" and waits for you to type `yes`, which would hang the script. This option accepts a first-time connection automatically.

> **WHOSE SHELL FILLS IN THE VARIABLES?**
>
> The variables here (like `$FULL_NAME`) are filled in by **your local shell** before the command is sent, because the SSH block uses double quotes. That is what we want — the real image name gets baked in and sent to the server. Just be aware of the rule: with double quotes, `$VAR` is expanded locally; if you ever need a variable that only exists *on* the server, you would use single quotes so it is left alone. This is the single most common source of confusion with remote SSH scripts.

5.3 Run it and see your app live

```
chmod +x deploy.sh
./deploy.sh
```

Open a browser to `http://<EC2_PUBLIC_IP>:4200`. Your application is running on a server on the internet, deployed by a single command from your own machine. The whole release is now two commands: `./build-and-push.sh` then `./deploy.sh` — a complete, if simple, deployment pipeline.

> **WHERE THIS GOES NEXT**
>
> Because deploy now runs entirely from your machine with no manual server login, a pipeline can run the exact same script for you. A tool like GitHub Actions would run Script 1 when you push code, then run Script 2 — the same `ssh ec2-user@... "..."` you just wrote — so a `git push` deploys the app with no manual steps. That is the direction the rest of the course builds toward.

6 Exercises

Each exercise gives you a problem to reproduce, then asks you to fix it. Use the material above and the bash and Docker references at the end.

A The script that lies about succeeding

PROBLEM

Write a build-and-push script *without* the `set -euo pipefail` line. Break the build on purpose (a typo in the Dockerfile), and run the script.

The build fails, but the script pushes anyway and prints "Done" — publishing a broken or stale image.

YOUR TASK

Explain why the script continued after the failure, then make it stop on the first error.

Hint. Section 2.3. What does bash do by default when a command in a script fails, and which line changes that behaviour?

B Access denied on push

PROBLEM

Tag your image as `mern-server:v1` (no username) and try to push it.

The push is rejected with `requested access to the resource is denied`.

YOUR TASK

Explain why Docker Hub rejects it, and push it successfully.

Hint. Section 3.1. How must an image be named for Docker Hub to accept it, and have you logged in?

C The key that is too open

PROBLEM

Try to SSH into your EC2 instance immediately after downloading the `.pem` key, without changing its permissions.

SSH refuses to connect, warning that the private key is too open / unprotected.

YOUR TASK

Explain what SSH is objecting to, and connect successfully.

Hint. This is the Week 1 permissions lesson on a key file. Which chmod value makes a file readable only by you?

D The app that is running but unreachable

PROBLEM

Deploy successfully with `deploy.sh`. Checking the server with `ssh -i my-key.pem ec2-user@<IP> "docker ps"` shows the container up — but `http://<EC2_PUBLIC_IP>:4200` in your browser times out.

YOUR TASK

The container is fine, so the problem is elsewhere. Find it and make the app reachable.

Hint. Two layers publish a port now, not one. The container is published to the server with -p; but what about the server itself? Re-read Section 4.1 — what does the EC2 security group allow in?

E The redeploy that fails the second time

PROBLEM

Take the deploy script and remove the `|| true` from the stop/remove lines. Run it twice from your machine.

The first run works. Something about running it a second time behaves differently from the first — investigate what.

YOUR TASK

Explain why the two runs differ, and make the script safe to run repeatedly.

Hint. *Section 5.2. On the first run, is there an old container to stop? On the second? What does set -e do when docker stop fails?*

7 When It Goes Wrong

Symptom	Cause and fix
Script continues after a command fails	Missing set -euo pipefail. Add it near the top so the script stops on the first error.
permission denied when running ./script.sh	The file is not executable. Run chmod +x script.sh first (Week 1 permissions).
denied: requested access ... on push	Image not named USER/name, or not logged in. Tag with your username and run docker login (Section 3).
SSH: Permissions ... are too open for key	The .pem key is world-readable. Run chmod 400 my-key.pem.
Container runs on EC2 but app unreachable	The EC2 security group does not allow the app's port. Add an inbound rule for it (Section 4.1).
deploy.sh hangs on first connection	SSH is waiting for you to confirm a new host. Add -o StrictHostKeyChecking=accept-new to the ssh command (Section 5.2).
deploy.sh fails on first run at docker stop	There is no old container yet. Append || true so the script continues (Section 5.2).

8 Before Next Week

Check you can do each of these without looking anything up.

1. Write a bash script with a shebang, `set -euo pipefail`, and a variable, and run it.
2. Explain why `set -euo pipefail` matters in an automation script.
3. Name and push an image to Docker Hub, and explain the `USER/name:tag` convention.
4. Launch an EC2 instance, install Docker, and connect over SSH with a key.
5. Deploy your image to EC2 with a single script run from your own machine, and reach the app in a browser.

Looking ahead: your deploy already runs from your machine over SSH with no manual server login — so a CI/CD pipeline can run the very same scripts. Next you will connect them so that pushing your code triggers the build, push, and deploy automatically.

Reference

— Bash guide — `linuxconfig.org/bash-scripting-tutorial`
— Push to Docker Hub — `docs.docker.com/docker-hub/repos/manage/hub-images/push/`
— Install Docker on Amazon Linux 2023 — `docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html`
— Set up WSL2 on Windows — `learn.microsoft.com/windows/wsl/install`